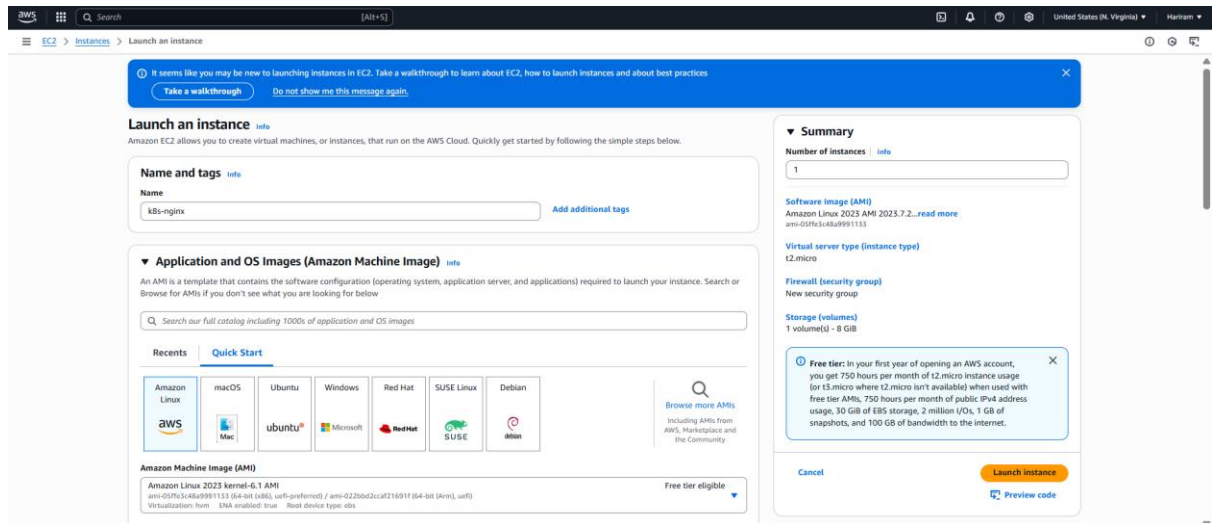
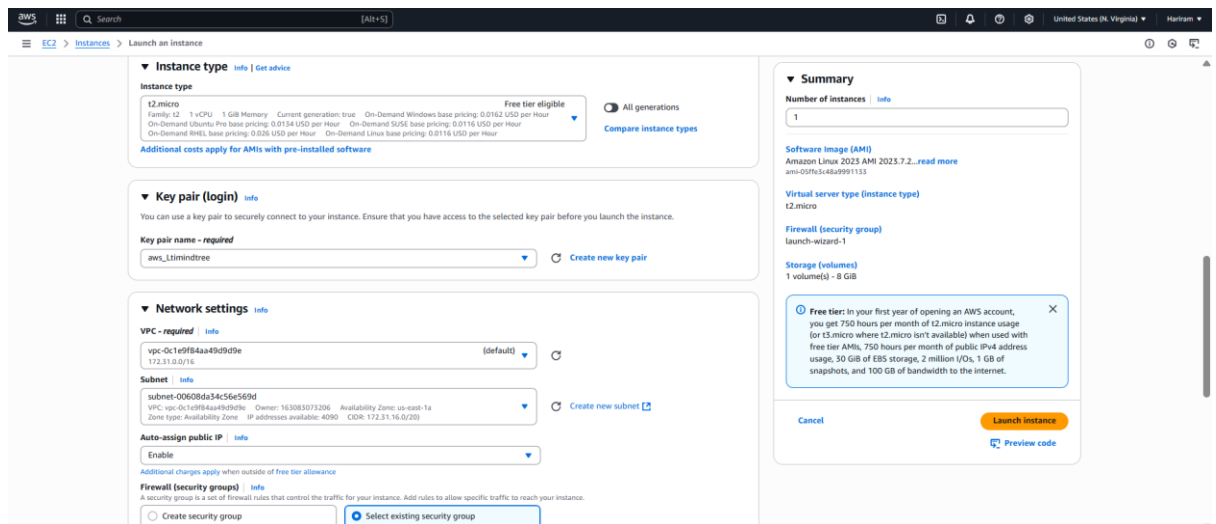


1) Easy

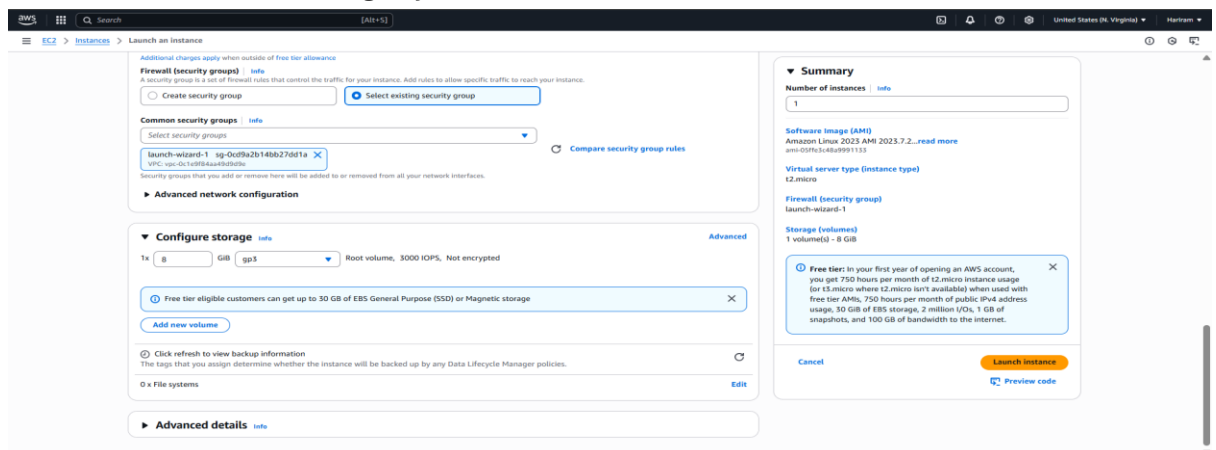
Deploy a Nginx application on your kubernetes cluster. and it should be available across of the cluster on port NO. 80.



Created a instance with “k8s-nginx” name. Amazon Linux is chosen.



T2.micro selected. Existing key is added. Subnet us-east-1a is chosen.



Existing security group is created. 8GB size is selected.


```
root@ip-172-31-27-61:~# You can now run: /usr/local/bin/aws --version
root@eks ~# aws configure
AWS Access Key ID [None]: AKIASLGECG23P6NHV2MA
AWS Secret Access Key [None]: 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
Default region name [None]: us-east-1
Default output format [None]: table
root@eks ~# curl -s -silent --location "https://github.com/weaveworks/eksctl/releases/latest/download/eksctl_${uname -s}_amd64.tar.gz" | tar xz -C /tmp
root@eks ~# sudo mv /tmp/eksctl /usr/local/bin
root@eks ~# eksctl version
0.210.0
root@eks ~# curl -LO https://storage.googleapis.com/kubernetes-release/release/$(curl -s https://storage.googleapis.com/kubernetes-release/release/stable.txt)/bin/linux/amd64/kubectl
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed

100 53.7M 100 53.7M    0     0  92.8M    0 --:--:-- --:--:-- --:--:--  92.8M
root@eks ~# sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
root@eks ~# kubectl version --client
Client Version: v1.31.0
Kustomize Version: v5.4.2
root@eks ~# ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_rsa
Your public key has been saved in /root/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:KfFwbPQCg9owEfxSbm4nLFAqkNL8htBSZJFw5vJ2T0 root@eks.example.com
The key's randomart image is:
+---[RSA 3072]-----+
|.O.o.|
|.o.o.o.|
|oOoo--|
|oOo+oP |
|*..o%=o.S|
|+..O.o.E|
|...|
+---[SHA256]-----+
root@eks ~#
```

Kubernetes is installed and ssh key is generated.

```
root@ip-172-31-27-61:~# [root@eks ~]# eksctl create cluster --name my-cluster-hari --region us-east-1 --version 1.32 --vpc-public-subnets subnet-00608da34c56e569d,subnet-0541b7ff15bd92fd9 --with-out-nodegroup
2025-07-05 10:52:34 [I] eksctl version 0.210.0
2025-07-05 10:52:34 [I] using region us-east-1
2025-07-05 10:52:34 [I] using existing VPC (vpc-0c1e9f84aa49d9d9e) and subnets (private:map[] public:map[us-east-1a:[subnet-00608da34c56e569d us-east-1a 172.31.16.0/20 0 ] us-east-1b:[subnet-0541b7ff15bd92fd9 us-east-1b 172.31.32.0/20 0 ]])
2025-07-05 10:52:34 [I] custom VPC/subnets will be used; if resulting cluster doesn't function as expected, make sure to review the configuration of VPC/subnets
2025-07-05 10:52:34 [I] using Kubernetes version 1.32
2025-07-05 10:52:34 [I] creating EKS cluster "my-cluster-hari" in "us-east-1" region with
2025-07-05 10:52:34 [I] if you encounter any issues, check CloudFormation console or try 'eksctl utils describe-stacks --region=us-east-1 --cluster=my-cluster-hari'
2025-07-05 10:52:34 [I] Kubernetes API endpoint access will use default of {publicAccess=true, privateAccess=false} for cluster "my-cluster-hari" in "us-east-1"
2025-07-05 10:52:34 [I] CloudWatch logging will not be enabled for cluster "my-cluster-hari" in "us-east-1"
2025-07-05 10:52:34 [I] you can enable it with 'eksctl utils update-cluster-logging --enable-types={SPECIFY-YOUR-LOG-TYPES-HERE (e.g. all)} --region=us-east-1 --cluster=my-cluster-hari'
2025-07-05 10:52:34 [I] default addons metrics-server, vpc-cni, kube-proxy, coredns were not specified, will install them as EKS addons
2025-07-05 10:52:34 [I]
2 sequential tasks: { create cluster control plane "my-cluster-hari",
2 sequential sub-tasks: {
1 task: { create addons },
wait for control plane to become ready,
}
}
2025-07-05 10:52:34 [I] building cluster stack "eksctl-my-cluster-hari-cluster"
2025-07-05 10:52:35 [I] deploying stack "eksctl-my-cluster-hari-cluster"
2025-07-05 10:53:05 [I] waiting for CloudFormation stack "eksctl-my-cluster-hari-cluster"
2025-07-05 10:53:35 [I] waiting for CloudFormation stack "eksctl-my-cluster-hari-cluster"
2025-07-05 10:54:35 [I] waiting for CloudFormation stack "eksctl-my-cluster-hari-cluster"
2025-07-05 10:55:35 [I] waiting for CloudFormation stack "eksctl-my-cluster-hari-cluster"
2025-07-05 10:56:35 [I] waiting for CloudFormation stack "eksctl-my-cluster-hari-cluster"
2025-07-05 10:57:35 [I] waiting for CloudFormation stack "eksctl-my-cluster-hari-cluster"
2025-07-05 10:58:35 [I] waiting for CloudFormation stack "eksctl-my-cluster-hari-cluster"
2025-07-05 10:59:35 [I] waiting for CloudFormation stack "eksctl-my-cluster-hari-cluster"
2025-07-05 11:00:35 [I] waiting for CloudFormation stack "eksctl-my-cluster-hari-cluster"
2025-07-05 11:01:35 [I] waiting for CloudFormation stack "eksctl-my-cluster-hari-cluster"
2025-07-05 11:01:36 [I] creating addon: metrics-server
2025-07-05 11:01:37 [I] successfully created addon: metrics-server
2025-07-05 11:01:37 [I] recommended policies were found for "vpc-cni" addon, but since OIDC is disabled on the cluster, eksctl cannot configure the requested permissions
, the recommended way to provide IAM permissions for "vpc-cni" addon is via pod identity associations; after addon creation is completed, add all recommended policies to
the config file, under 'addon.PodIdentityAssociations', and run 'eksctl update addon'
2025-07-05 11:01:37 [I] creating addon: vpc-cni
2025-07-05 11:01:37 [I] successfully created addon: vpc-cni
2025-07-05 11:01:38 [I] creating addon: kube-proxy
2025-07-05 11:01:38 [I] successfully created addon: kube-proxy
2025-07-05 11:01:38 [I] creating addon: coredns
2025-07-05 11:01:38 [I] successfully created addon: coredns
```

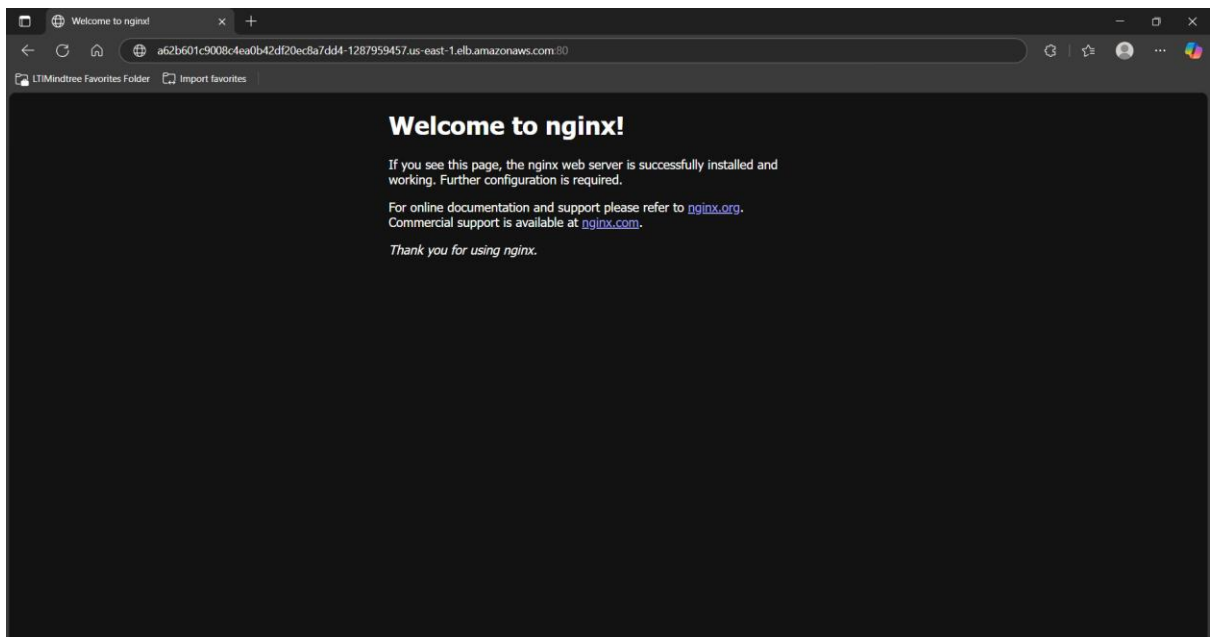
Cluster is created successfully.

```
root@ip-172-31-27-61:~# [root@eks ~]# eksctl create nodegroup --cluster my-cluster-hari --region us-east-1 --name my-node-group --node-ami-family Ubuntu2204 --node-type t2.small --su-bnet-ids subnet-00608da34c56e569d,subnet-0541b7ff15bd92fd9 --nodes 3 --nodes-min 2 --nodes-max 4 --ssh-access --ssh-public-key /root/.ssh/id_rsa.pub
2025-07-05 11:06:44 [I] will use version 1.32 for new nodegroup(s) based on control plane version
2025-07-05 11:06:44 [I] nodegroup "my-node-group" will use "ami-08765d7d77f5892000" [Ubuntu2204/1.32]
2025-07-05 11:06:44 [I] using SSH public key "/root/.ssh/id_rsa.pub" as "eksctl-my-cluster-hari-nodegroup-my-node-group-db:33:73:b0:4b:03:46:23:f6:22:e8:5a:07:b8:f2:1a"
2025-07-05 11:06:44 [I] 1 nodegroup (my-node-group) was included (based on the include/exclude rules)
2025-07-05 11:06:44 [I] will create a CloudFormation stack for each of 1 managed nodegroups in cluster "my-cluster-hari"
2025-07-05 11:06:44 [I]
2 sequential tasks: { fix cluster compatibility, 1 task: { 1 task: { create managed nodegroup "my-node-group" } }
}
2025-07-05 11:06:44 [I] checking cluster stack for missing resources
2025-07-05 11:06:44 [I] cluster stack has all required resources
2025-07-05 11:06:44 [I] building managed nodegroup stack "eksctl-my-cluster-hari-nodegroup-my-node-group"
2025-07-05 11:06:45 [I] deploying stack "eksctl-my-cluster-hari-nodegroup-my-node-group"
2025-07-05 11:06:45 [I] waiting for CloudFormation stack "eksctl-my-cluster-hari-nodegroup-my-node-group"
2025-07-05 11:07:15 [I] waiting for CloudFormation stack "eksctl-my-cluster-hari-nodegroup-my-node-group"
2025-07-05 11:07:51 [I] waiting for CloudFormation stack "eksctl-my-cluster-hari-nodegroup-my-node-group"
2025-07-05 11:08:00 [I] waiting for CloudFormation stack "eksctl-my-cluster-hari-nodegroup-my-node-group"
2025-07-05 11:08:15 [I] waiting for CloudFormation stack "eksctl-my-cluster-hari-nodegroup-my-node-group"
2025-07-05 11:11:00 [I] waiting for CloudFormation stack "eksctl-my-cluster-hari-nodegroup-my-node-group"
2025-07-05 11:11:00 [I] no tasks
2025-07-05 11:11:00 [I] created 0 nodegroup(s) in cluster "my-cluster-hari"
2025-07-05 11:11:00 [I] nodegroup "my-node-group" has 3 node(s)
2025-07-05 11:11:00 [I] node "ip-172-31-19-207.ec2.internal" is ready
2025-07-05 11:11:00 [I] node "ip-172-31-26-122.ec2.internal" is ready
2025-07-05 11:11:00 [I] node "ip-172-31-47-128.ec2.internal" is ready
2025-07-05 11:11:00 [I] waiting for at least 2 node(s) to become ready in "my-node-group"
2025-07-05 11:11:00 [I] nodegroup "my-node-group" has 3 node(s)
2025-07-05 11:11:00 [I] node "ip-172-31-19-207.ec2.internal" is ready
2025-07-05 11:11:00 [I] node "ip-172-31-26-122.ec2.internal" is ready
2025-07-05 11:11:00 [I] node "ip-172-31-47-128.ec2.internal" is ready
2025-07-05 11:11:00 [I] created 1 managed nodegroup(s) in cluster "my-cluster-hari"
2025-07-05 11:11:00 [I] checking security group configuration for all nodegroups
2025-07-05 11:11:00 [I] all nodegroups have up-to-date cloudformation templates
root@eks ~#
```

Node group is created successfully.


```
root@ip-172-31-27-61:~  
[root@eks ~]# vim nginx.yaml  
[root@eks ~]# vim nginx-service.yaml  
[root@eks ~]# kubectl apply -f nginx.yaml  
deployment.apps/nginx-deployment created  
[root@eks ~]# kubectl apply -f nginx-service.yaml  
service/nginx-service created  
[root@eks ~]# kubectl get nodes  
NAME                                STATUS    ROLES    AGE     VERSION  
ip-172-31-19-207.ec2.internal      Ready    <none>   6m49s   v1.32.3  
ip-172-31-26-122.ec2.internal      Ready    <none>   6m49s   v1.32.3  
ip-172-31-47-128.ec2.internal      Ready    <none>   6m49s   v1.32.3  
[root@eks ~]# kubectl get pods  
NAME                                READY    STATUS    RESTARTS   AGE  
nginx-deployment-96b9d695-2dl2q    1/1     Running   0           26s  
nginx-deployment-96b9d695-2ptl7    1/1     Running   0           25s  
nginx-deployment-96b9d695-bptnv    1/1     Running   0           26s  
[root@eks ~]# kubectl get svc  
NAME            TYPE           CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE  
kubernetes      ClusterIP      10.100.0.1       <none>           443/TCP          18m  
nginx-service   LoadBalancer  10.100.106.98   a62b601c9008c4ea0b42df20ec8a7dd4-1287959457.us-east-1.elb.amazonaws.com  80:32087/TCP    23s  
[root@eks ~]# |
```

Both yaml is applied using kubectl apply -f command. Pods and nodes are running. Using kubectl get svc command external ip is taken.

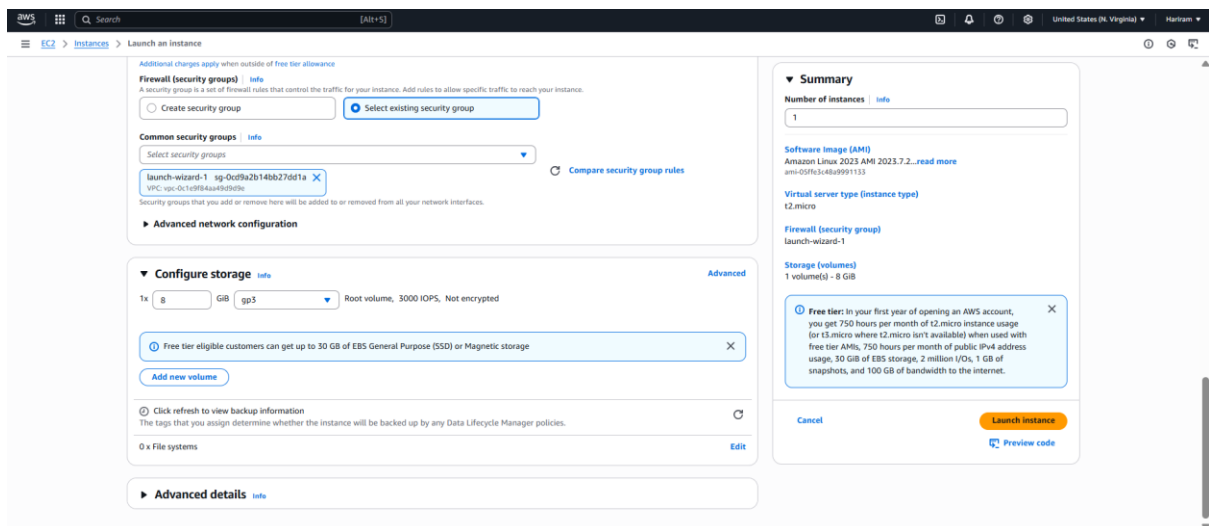


External ip with port 80 is pasted in the web-browser , Finally nginx web-page is available.

Create an EC2 instance using of terraform us-east-1a zone in North Virginia region. And create of your security group which name is web_access and allow port 22 and 80 in the ingress. This security group should be attached with your EC2 instance via Terraform.

Created instance with name as "Terraform-EC2". Amazon Linux is chosen.

Existing key is selected. Subnet is selected. Existing security group is selected.



8GB size is selected and gp3 is selected.

```
root@ip-172-31-27-88/ec2 ~$ ssh -i "aws_ltmindtree.pem" ec2-user@ec2-18-209-5-223.compute-1.amazonaws.com
PS C:\Users\10835505\OneDrive - LTIMindtree\Desktop> ssh -i "aws_ltmindtree.pem" ec2-user@ec2-18-209-5-223.compute-1.amazonaws.com
The authenticity of host 'ec2-18-209-5-223.compute-1.amazonaws.com (18.209.5.223)' can't be established.
ED25519 key fingerprint is SHA256:olbLYjIxlwyOglwyCieIR4CLWfys4X5Szxnz2YRNRw.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ec2-18-209-5-223.compute-1.amazonaws.com' (ED25519) to the list of known hosts.

      _#_
     _###_
    _####_
   _####_
  _####_
 _####_
_####_

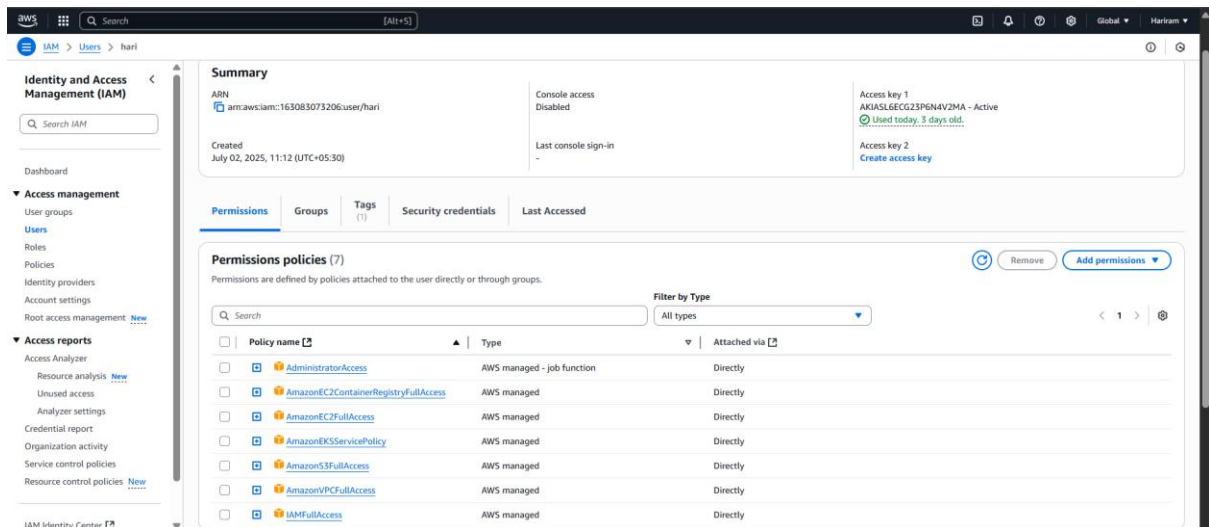
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

[ec2-user@ip-172-31-27-88 ~]$ sudo su -
[root@ip-172-31-27-88 ~]# hostnamectl set-hostname terraform.example.com
[root@ip-172-31-27-88 ~]# bash
[root@terraform ~]# cd /
[root@terraform /]# mkdir /ec2
[root@terraform /]# cd /ec2/
[root@terraform ec2]# ll
total 0
[root@terraform ec2]#
```

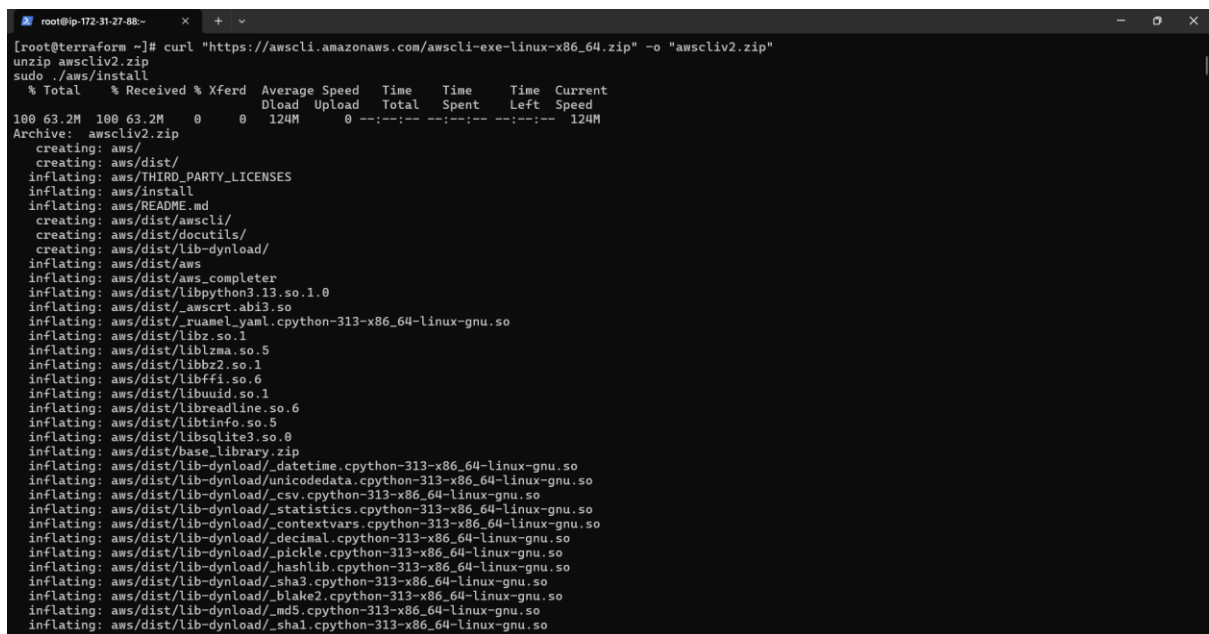
Machine is running successfully. Inside root folder ec2 directory is created.

```
root@ip-172-31-27-88:~$ sudo dnf install -y dnf-plugins-core
Amazon Linux 2023 Kernel Livepatch repository
Package dnf-plugins-core-4.3.0-13.amzn2023.0.5.noarch is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[root@terraform ~]# sudo dnf config-manager --add-repo https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
Adding repo from: https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
[root@terraform ~]# sudo dnf -y install terraform
Hashicorp Stable - x86_64
Last metadata expiration check: 0:00:01 ago on Sat Jul 5 10:59:35 2025.
Dependencies resolved.
=====
Package                               Architecture      Version           Repository        Size
=====
Installing:
terraform                               x86_64            1.12.2-1          hashicorp          28 M
Installing dependencies:
git                                     x86_64            2.47.1-1.amzn2023.0.3  amazonlinux       52 k
git-core                               x86_64            2.47.1-1.amzn2023.0.3  amazonlinux       4.5 M
git-core-doc                           noarch            2.47.1-1.amzn2023.0.3  amazonlinux       2.8 M
perl-Error                             noarch            1:0.17029-5.amzn2023.0.2  amazonlinux       41 k
perl-File-Find                         noarch            1.37-477.amzn2023.0.7  amazonlinux       25 k
perl-Git                               noarch            2.47.1-1.amzn2023.0.3  amazonlinux       40 k
perl-TermReadKey                       x86_64            2.38-9.amzn2023.0.2  amazonlinux       36 k
perl-lib                               x86_64            0.65-477.amzn2023.0.7  amazonlinux       15 k
=====
Transaction Summary
-----
Install 9 Packages
Total download size: 36 M
Installed size: 126 M
Downloading Packages:
(1/9): git-2.47.1-1.amzn2023.0.3.x86_64.rpm           1.4 MB/s | 52 kB  00:00
(2/9): perl-Error-0.17029-5.amzn2023.0.2.noarch.rpm    1.8 MB/s | 41 kB  00:00
(3/9): git-core-doc-2.47.1-1.amzn2023.0.3.noarch.rpm   32 MB/s | 2.8 MB  00:00
(4/9): perl-File-Find-1.37-477.amzn2023.0.7.noarch.rpm  835 kB/s | 25 kB  00:00
(5/9): git-core-2.47.1-1.amzn2023.0.3.x86_64.rpm      37 MB/s | 4.5 MB  00:00
(6/9): perl-Git-2.47.1-1.amzn2023.0.3.noarch.rpm       1.1 MB/s | 40 kB  00:00
(7/9): perl-TermReadKey-2.38-9.amzn2023.0.2.x86_64.rpm 1.0 MB/s | 36 kB  00:00
```

Terraform is installed successfully.



IAM user is created with necessary permissions.



AWS cli is installed.



AWS cli is configured successfully.



Ec2.tf file is created.


```

root@ip-172-31-27-88/ec2 x + v
resource "aws_security_group" "web_access" {
  name      = "web_access"
  description = "Allow traffic"

  tags = {
    Name = "web_access"
  }
}

resource "aws_vpc_security_group_ingress_rule" "allow_http" {
  security_group_id = aws_security_group.web_access.id
  cidr_ipv4         = "0.0.0.0/0"
  from_port         = 80
  ip_protocol       = "tcp"
  to_port           = 80
}

resource "aws_vpc_security_group_ingress_rule" "allow_ssh" {
  security_group_id = aws_security_group.web_access.id
  cidr_ipv4         = "0.0.0.0/0"
  from_port         = 22
  ip_protocol       = "tcp"
  to_port           = 22
}

resource "aws_vpc_security_group_egress_rule" "allow_all_traffic" {
  security_group_id = aws_security_group.web_access.id
  cidr_ipv4         = "0.0.0.0/0"
  ip_protocol       = "-1"
}

#ec2 instance code starts here
resource "aws_instance" "custom-server" {
  ami              = "ami-05ffe3c48a9991133"
  availability_zone = "us-east-1a"
  instance_type    = "t2.micro"
  security_groups  = ["${aws_security_group.web_access.name}"]
  key_name         = "aws_1timindtree"
  root_block_device {
    volume_size = "25"
    volume_type = "gp2"
    delete_on_termination = true
  }
}
-- INSERT --
1,1 Top

```

Yaml code is written such that EC2 instance will be created and a new security group will be created with port 80 ,22 allowed and finally the security group will be attached to the instance.

```

root@ip-172-31-27-88/ec2 x + v
[root@terraform /]# cd /
[root@terraform /]# cd /ec2/
[root@terraform ec2]# vim ec2.tf
[root@terraform ec2]# terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.2.0...
- Installed hashicorp/aws v6.2.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
[root@terraform ec2]# terraform fmt
ec2.tf
[root@terraform ec2]# terraform validate
Success! The configuration is valid.
[root@terraform ec2]#

```

Terraform is initiated successfully inside the ec2 directory. Terraform fmt is done.

Terraform validate is successfully.

```
root@ip-172-31-27-88/ec2 x + v
+ region          = "us-east-1"
+ security_group_id = (known after apply)
+ security_group_rule_id = (known after apply)
+ tags_all        = {}
+ to_port         = 80
}

# aws_vpc_security_group_ingress_rule.allow_https will be created
+ resource "aws_vpc_security_group_ingress_rule" "allow_https" {
+   arn          = (known after apply)
+   cidr_ipv4    = "0.0.0.0/0"
+   from_port    = 443
+   id           = (known after apply)
+   ip_protocol  = "tcp"
+   region       = "us-east-1"
+   security_group_id = (known after apply)
+   security_group_rule_id = (known after apply)
+   tags_all     = {}
+   to_port      = 443
}

# aws_vpc_security_group_ingress_rule.allow_ssh will be created
+ resource "aws_vpc_security_group_ingress_rule" "allow_ssh" {
+   arn          = (known after apply)
+   cidr_ipv4    = "0.0.0.0/0"
+   from_port    = 22
+   id           = (known after apply)
+   ip_protocol  = "tcp"
+   region       = "us-east-1"
+   security_group_id = (known after apply)
+   security_group_rule_id = (known after apply)
+   tags_all     = {}
+   to_port      = 22
}

Plan: 6 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
[root@terraform ec2]#
```

Terraform plan is successfully.

```
root@ip-172-31-27-88/ec2 x + v
# aws_vpc_security_group_ingress_rule.allow_ssh will be created
+ resource "aws_vpc_security_group_ingress_rule" "allow_ssh" {
+   arn          = (known after apply)
+   cidr_ipv4    = "0.0.0.0/0"
+   from_port    = 22
+   id           = (known after apply)
+   ip_protocol  = "tcp"
+   region       = "us-east-1"
+   security_group_id = (known after apply)
+   security_group_rule_id = (known after apply)
+   tags_all     = {}
+   to_port      = 22
}

Plan: 6 to add, 0 to change, 0 to destroy.

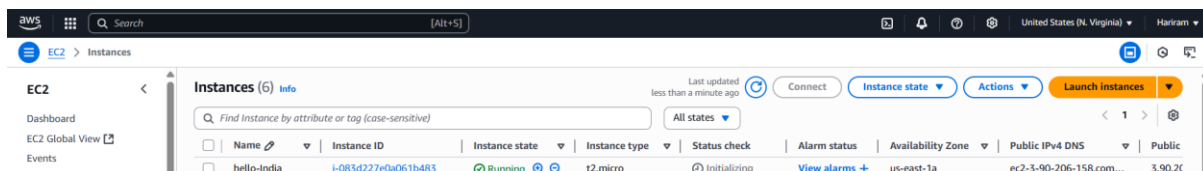
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_security_group.web_access: Creating...
aws_security_group.web_access: Creation complete after 2s [id=sg-032cbdf840fc08317]
aws_instance.custom-server: Creating...
aws_vpc_security_group_ingress_rule.allow_https: Creating...
aws_vpc_security_group_egress_rule.allow_all_traffic: Creating...
aws_vpc_security_group_ingress_rule.allow_ssh: Creating...
aws_vpc_security_group_ingress_rule.allow_http: Creating...
aws_vpc_security_group_ingress_rule.allow_https: Creation complete after 0s [id=sgr-0acb5fa322c558e42]
aws_vpc_security_group_egress_rule.allow_all_traffic: Creation complete after 0s [id=sgr-0b7d2679ccad4a8e]
aws_vpc_security_group_ingress_rule.allow_ssh: Creation complete after 0s [id=sgr-0ae2b667ce8f9c5d9]
aws_instance.custom-server: Still creating... [00m10s elapsed]
aws_instance.custom-server: Still creating... [00m20s elapsed]
aws_instance.custom-server: Still creating... [00m30s elapsed]
aws_instance.custom-server: Creation complete after 32s [id=i-083d227e0a061b483]

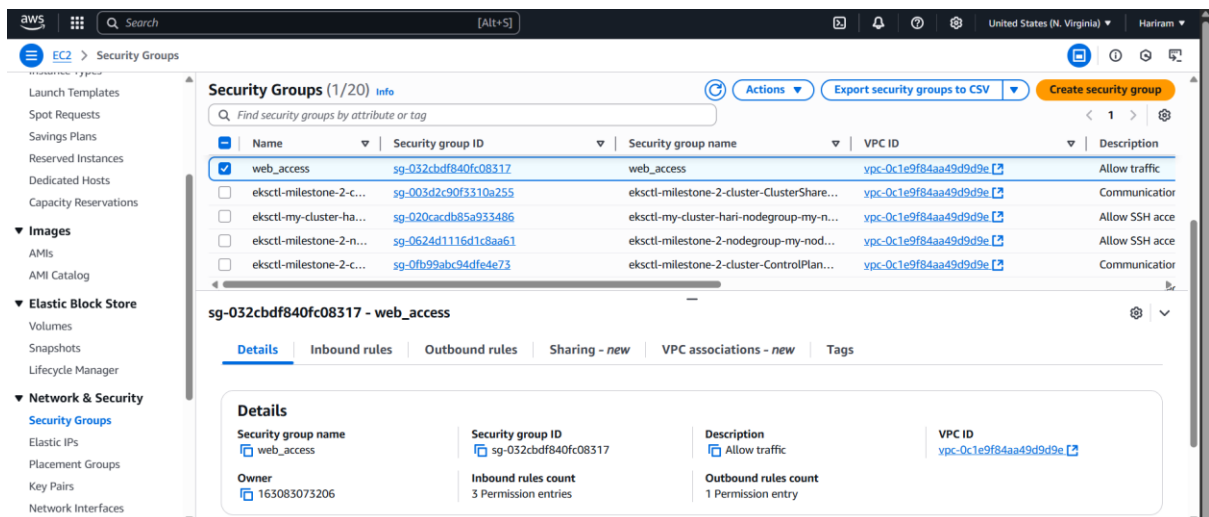
Apply complete! Resources: 6 added, 0 changed, 0 destroyed.
[root@terraform ec2]#
```

Terraform apply is successfully and new instance , security group is created and attached with the instance.

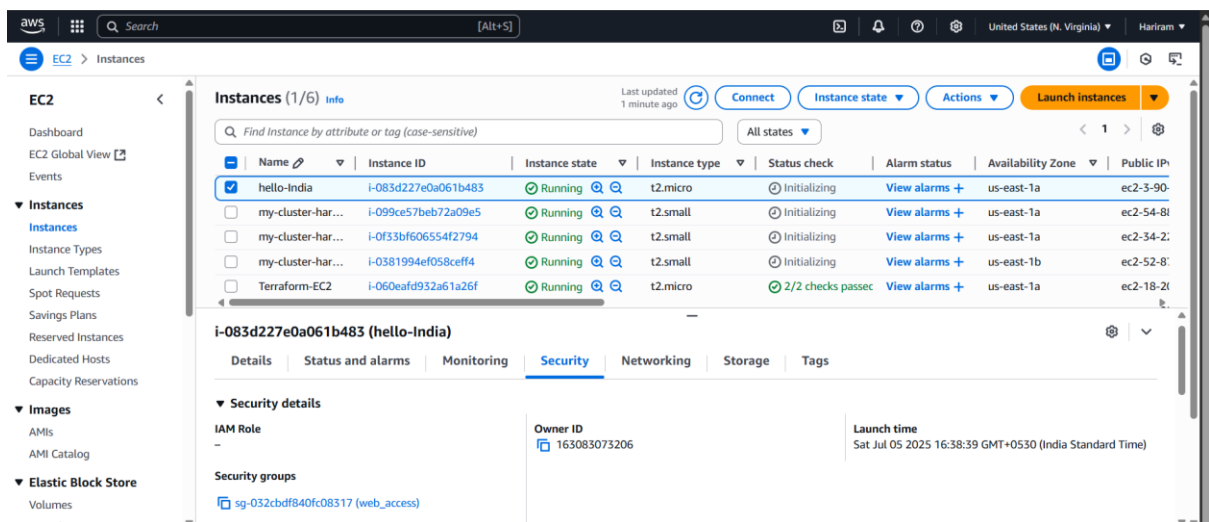


Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public
hello-india	i-083d227e0a061b483	Running	t2.micro	Initializing	View alarms +	us-east-1a	ec2-3-90-206-158.com...	3.90.206...

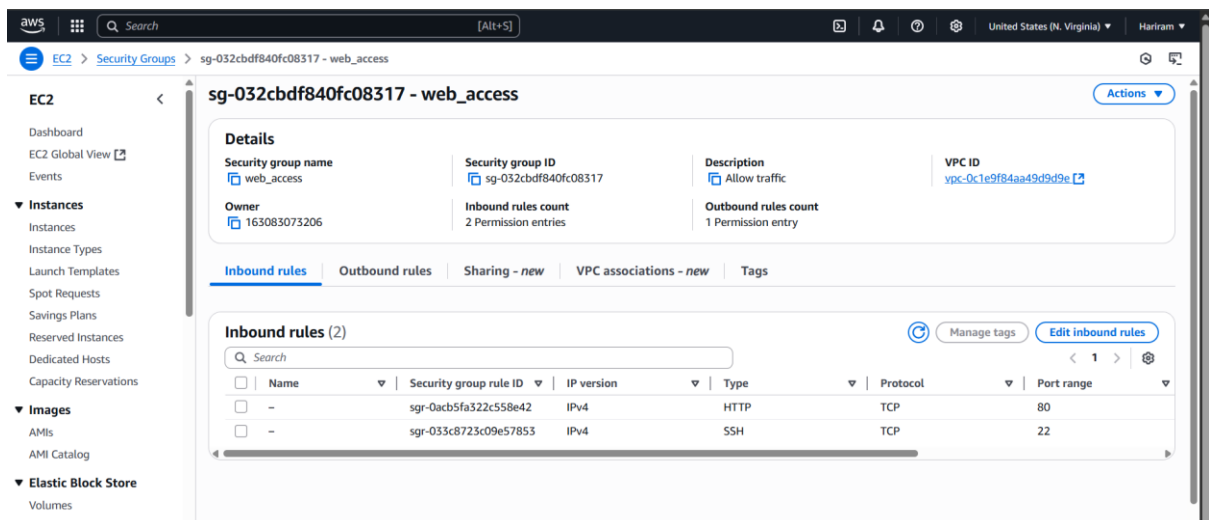
Instance is created successfully as hello-india.



Security group is created with name as “web_access”.



Security group is attached with the instance.



In the security group ports 80 and 22 are allowed.

